

CANTIN Alexis

GymBuddy

PARTIE A

Écrans et fonctionnalités développés

J'ai développé l'ensemble de l'application.

Ordre de développement

1. Bibliothèque d'exercices

J'ai commencé par cet écran, car il constitue le cœur de l'application grâce à l'API qui permet de récupérer les exercices. Chaque exercice possède une catégorie permettant de les organiser.

2. Création des programmes

J'ai ensuite développé la création de programmes en permettant d'ajouter différents exercices dans une liste pouvant être sauvegardée. J'ai également ajouté la possibilité de cliquer sur un exercice dans la bibliothèque afin d'afficher une fiche détaillée contenant sa description et de l'ajouter directement à un programme avec le nombre de séries, de répétitions et le poids cible.

3. Journal des séances

Lorsqu'une séance est lancée à partir d'un programme, il est possible de modifier le nombre de répétitions ainsi que le poids utilisé. J'ai également ajouté un minuteur pour le temps de repos avec une vibration à la fin du compte à rebours.

4. Historique des séances

Les séances sont enregistrées dans Supabase avec leur durée totale pour chaque programme. Un système de *Pull to Refresh* a également été ajouté.

5. Graphique de progression

J'ai développé un graphique affichant l'évolution du poids maximal par séance à l'aide d'une courbe de Bézier dessinée avec **react-native-svg**.

6. Authentification

L'inscription et la connexion par e-mail et mot de passe sont gérées avec **Supabase Auth**. Cette fonctionnalité a été développée en dernier afin d'éviter de devoir se reconnecter à chaque phase de test.

Difficultés techniques rencontrées

- Le projet a débuté avec le SDK 56, incompatible avec la version d'Expo Go disponible. La solution a consisté à revenir au SDK 54, supprimer le dossier **node_modules** puis réinstaller les dépendances avec l'option **--legacy-peer-deps** afin de résoudre les conflits de dépendances.
 - **victory-native** (version 40 et supérieure) nécessite **Skia** (build natif), tandis que **react-native-gifted-charts** nécessite **react-native-linear-gradient**, indisponible avec Expo Go. J'ai donc développé un graphique entièrement personnalisé avec **react-native-svg**, en utilisant **LinearGradient** en SVG afin d'éviter toute dépendance supplémentaire.
 - L'endpoint **/exercise/search/** renvoyait une erreur 404. La solution a été de charger 100 exercices par requête puis d'effectuer le filtrage côté client sur les traductions anglaises, avec un système de **debounce** pour limiter les appels réseau.
-

Ce que j'aurais fait différemment

J'aurais ajouté davantage de fonctionnalités, comme la modification des programmes, et j'aurais utilisé un peu moins l'intelligence artificielle.

PARTIE B

Outils utilisés

- **Claude Code** : pour le développement technique du projet.
 - **ChatGPT** : pour la correction du texte et la mise en page du PDF.
-

Exemples concrets de prompts

Pour un problème graphique

« Les boutons pour choisir la catégorie sont trop longs et ne sont pas adaptés à l'écran. »

Résultat :

Remplacement du **ScrollView** horizontal par un **View** utilisant **flexWrap: 'wrap'**, ce qui permet d'afficher toutes les catégories sans défilement horizontal.

Pour un problème technique

« Le projet actuel n'est pas compatible avec ma version d'Expo. Modifie-le pour le SDK 54. »

Résultat :

L'IA a identifié la cause du problème et effectué la correction complète des versions, des dépendances ainsi que leur réinstallation en une seule étape.

« Comment créer un APK Android ? »

Résultat :

Création du fichier **.apk** avec **EAS Build**, puis ajout de celui-ci au rendu.

Ce que l'IA a bien fait

- Maintien d'une cohérence graphique sur tous les écrans.
 - Diagnostic rapide des problèmes de compatibilité et de dépendances.
 - Gestion des erreurs (Supabase non configuré, données vides, erreurs réseau).
-

Ce que l'IA a moins bien fait

- La première bibliothèque de graphiques proposée n'était pas compatible avec Expo Go.
 - Certains composants générés étaient parfois trop volumineux et ont nécessité quelques ajustements ergonomiques.
-

Ma réflexion personnelle

L'intelligence artificielle m'a permis de gagner énormément de temps, notamment sur le design que j'ai pu lui déléguer presque entièrement. Elle m'a également aidé à résoudre rapidement les bugs et les erreurs. Au lieu d'effectuer de longues recherches sur Internet, je pouvais directement lui poser mes questions, ce qui m'a permis de progresser beaucoup plus efficacement.